

对话框与控件



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

主要内容

- 对话框的基本概念
 - 模态与非模态对话框
 - 控件的创建和使用方法
 - 通用对话框和消息对话框
-
- 对应课本：第九章

复习：模态与非模态对话框

➤ 模态对话框的创建和关闭

- 使用对话框类实例化对象
- 调用DoModal()成员函数创建和显示模态对话框
- 点击“确定”或“取消”按钮，或者在对话框类内部调用EndDialog()、OnOk()、OnCancel()函数关闭对话框

➤ 非模态对话框的创建和关闭

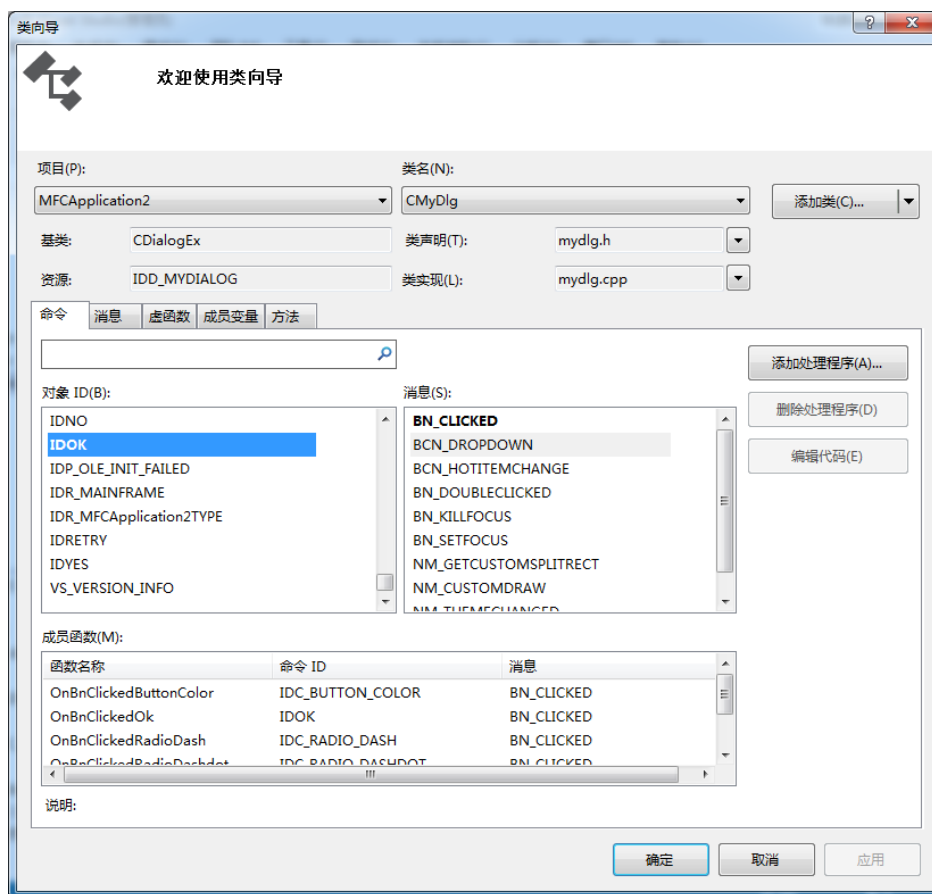
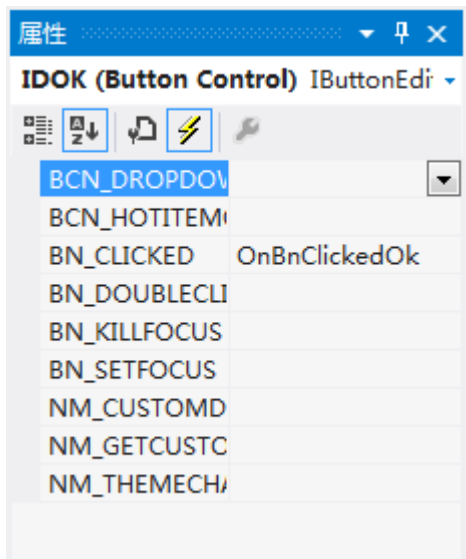
- 使用new运算符动态构造对话框对象
- 调用Create()成员函数创建对话框
- 调用ShowWindow()成员函数显示对话框
- 调用DestroyWindow()成员函数销毁对话框
- 使用delete运算符释放内存

复习：对话框的编程

1. 用对话框编辑器设计对话框
2. 用类向导创建对话框类
3. 用类向导或属性选项卡中的控件事件，为对话框中的控件与对话框类中的消息处理函数建立映射
4. 用类向导添加与控件相关成员变量
5. 编写控件的消息处理函数
6. 创建对话框模板并显示对话框

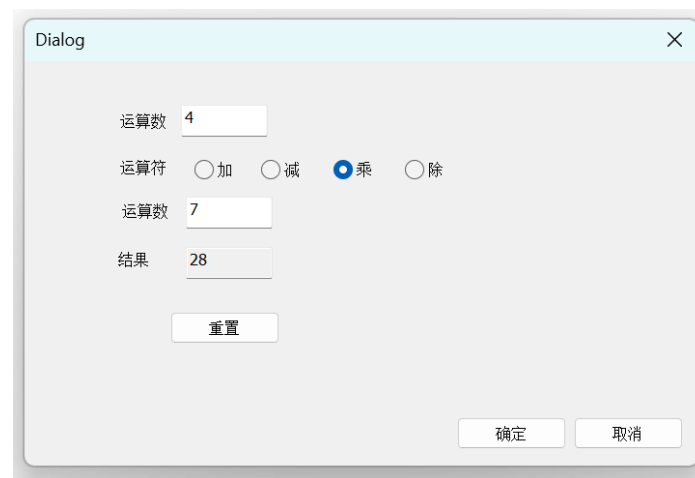
复习：控件的消息映射

- 为控件添加消息映射的方法
 - 方法一：使用属性选项卡中的控件事件
 - 方法二：使用类向导



复习：关键知识点

- Radio button Group
- 控件的顺序-Tab order (Ctrl+D)
- 控件的对齐



主要内容

- 对话框的基本概念
- 模态与非模态对话框
- 控件的创建和使用方法
- 通用对话框和消息对话框

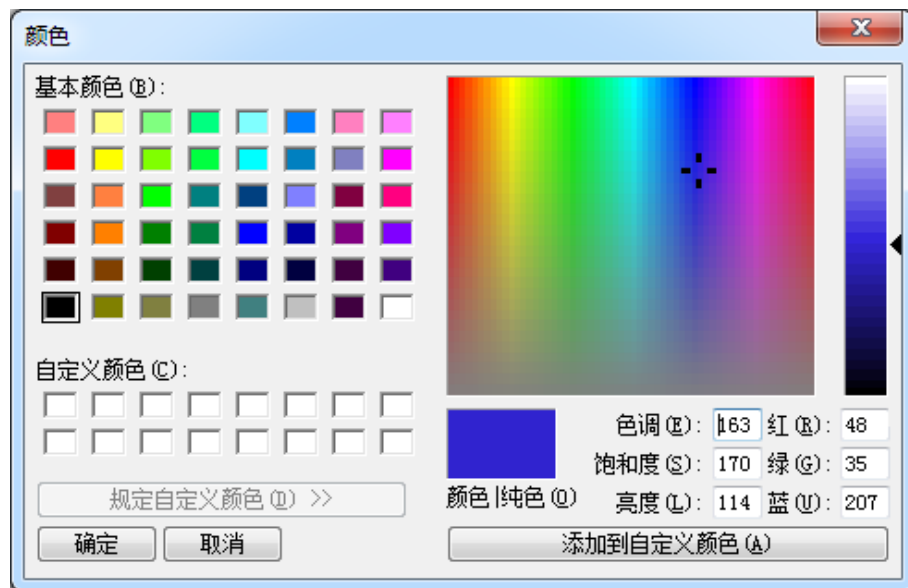
通用对话框

- 通用对话框是系统定义的对话框，为用户提供一组标准接口，通用对话框类都是从公共基类CCommonDialog派生而来的
- 可以使用通用对话框执行各种标准操作，如选择文件名（用于打开和保存文件）、选择字体、选择颜色、进行打印和打印设置、正文查找和替换等通用操作

对话框	用 途
CColorDialog	颜色对话框，允许用户选择或创建颜色
CFileDialog	文件对话框，允许用户打开或保存一个文件
CFindReplaceDialog	查找替换对话框，允许用户查找或替换指定字符串
CPageSetupDialog	页面设置对话框，允许用户设置页面参数
CFontDialog	字体对话框，允许用户从列出的可用字体中选择一种字体
CPrintDialog	打印对话框，允许用户设置打印机的参数及打印文档

1、CColorDialog类

- CColorDialog类封装了标准的颜色对话框
- 在Windows的画图程序中，如果用户在颜色面板的某种颜色上双击鼠标，就会显示一个编辑颜色对话框来让用户选择颜色
- 相关函数
 - GetColor: 获得颜色RGB值
 - GetSavedCustomColors: 获得自定义颜色
 - SetCurrentColor: 设置所选颜色



1、CColorDialog类

➤ 构造函数

系统程序

```
CColorDialog(  
    COLORREF clrInit = 0, // 默认颜色,为空则为RGB(0,0,0)(黑色)  
    DWORD dwFlags = 0, // 对话框属性  
    CWnd* pParentWnd = NULL ); // 父窗口
```

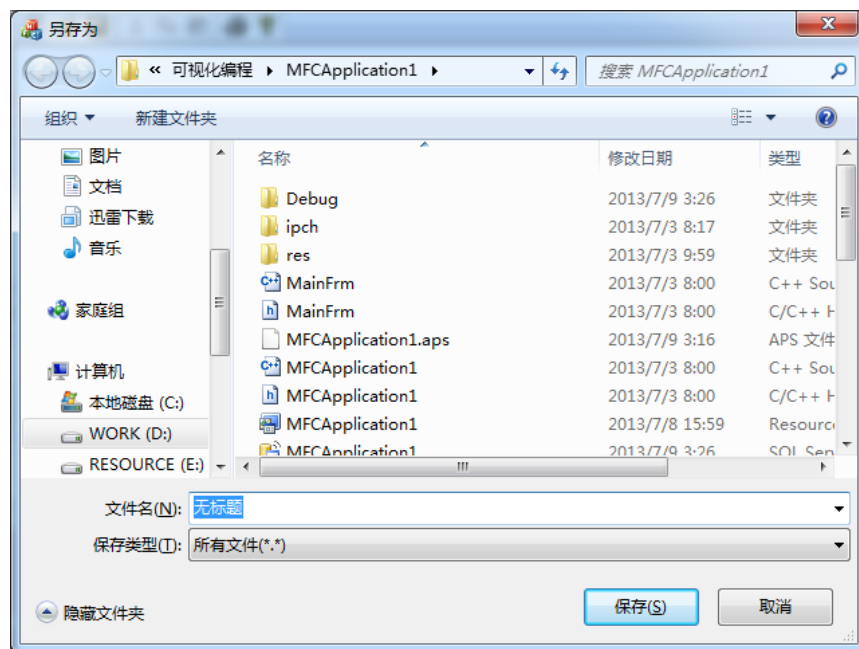
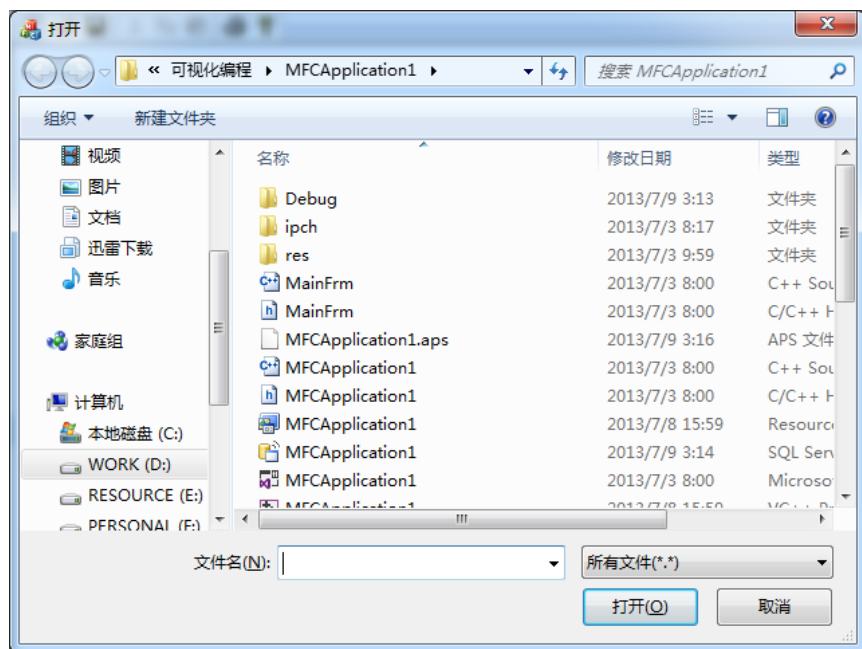
➤ 使用颜色对话框

菜单响应
函数

```
CClientDC dc(this);  
COLORREF color;  
CColorDialog colorDlg(RGB(255,255,0));  
if(colorDlg.DoModal() == IDOK){  
    color = colorDlg.GetColor();  
    dc.SetTextColor(color);  
    dc.TextOutW(100,100,_T("color test"));  
}
```

2、CFileDialog类

- CFileDialog类用于实现文件选择对话框，以支持文件的打开和保存操作
- 用MFC AppWizard建立的应用程序中自动加入了文件选择对话框，在File菜单选“打开”或“另存为”命令会启动它们



2、CFileDialog类

➤ 构造函数

系统程序

```
CFileDialog (  
    BOOL bOpenFileDialog,          // 打开(TRUE)或保存(FALSE)文件  
    LPCTSTR lpszDefExt = NULL,      // 缺省文件扩展名  
    LPCTSTR lpszFileName = NULL,    // 初始文件名  
    DWORD dwFlags = OFN_HIDEREADONLY |  
        OFN_OVERWRITEPROMPT,       // 对话框属性  
    LPCTSTR lpszFilter = NULL,      // 过滤字符串  
    CWnd* pParentWnd = NULL );     // 父窗口
```

- GetFileName: 获得文件名
- GetPathName: 获得文件路径
- GetFileExt: 获得文件扩展名
- GetStartPosition: 获得文件列表起始位置

2、CFileDialog类

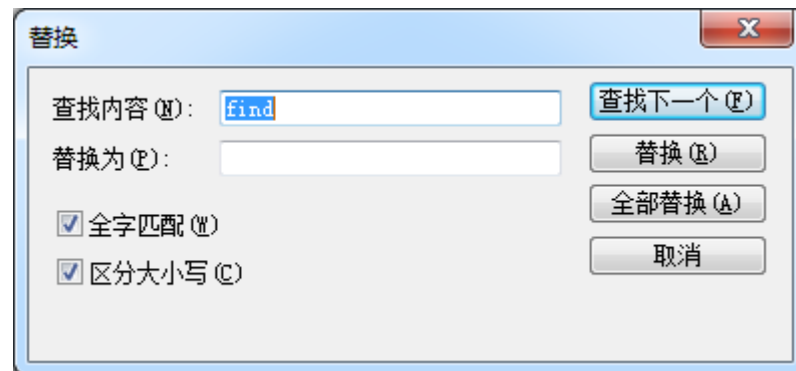
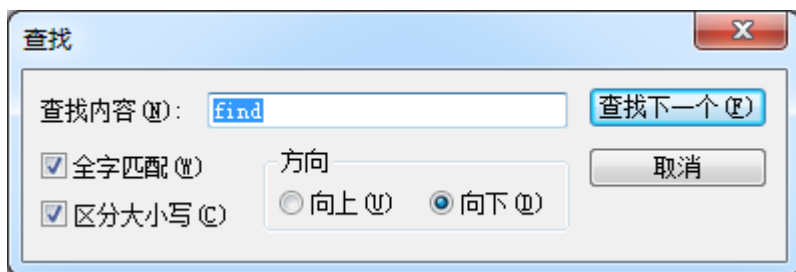
➤ 使用文件选择对话框

菜单响应
函数

```
CClientDC dc(this);  
CString FilePathName;  
  
CFileDialog fileDlg(FALSE);  
  
if(fileDlg.DoModal() == IDOK)  
{  
    FilePathName=fileDlg.GetPathName();  
    dc.TextOutW(100,100,FilePathName);  
}
```

3、CFindReplaceDialog类

- 查找替换对话框是无模式对话框



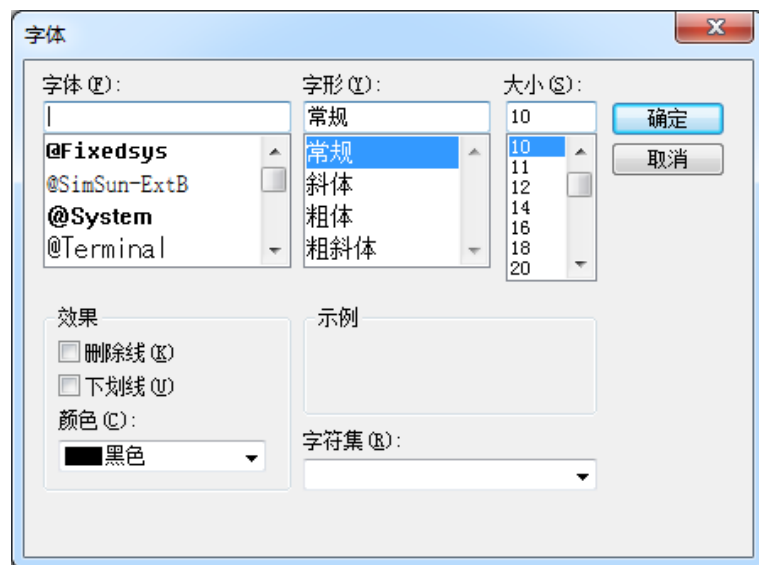
- 使用查找替换对话框

```
CFindReplaceDialog *pFindDlg = new CFindReplaceDialog;
if (!pFindDlg->Create(FALSE,_T("find"),NULL,
    FR_DOWN | FR_MATCHCASE | FR_WHOLEWORD ,this))
{
    return;
}
pFindDlg->ShowWindow(SW_SHOW);    //窗口显示
pFindDlg->SetActiveWindow();      //设置为活动窗口
```

菜单响应
函数

4、CFontDialog类

- 字体对话框相关函数
 - GetColor: 获得字体颜色
 - GetFaceName: 获得字体名字
 - GetCurrentFont: 获得选择的字体
 - IsBold: 字体是否为粗体
 - IsItalic: 字体是否为斜体
 - IsUnderline: 字体是否带下划线



4、CFontDialog类

➤ 使用字体对话框

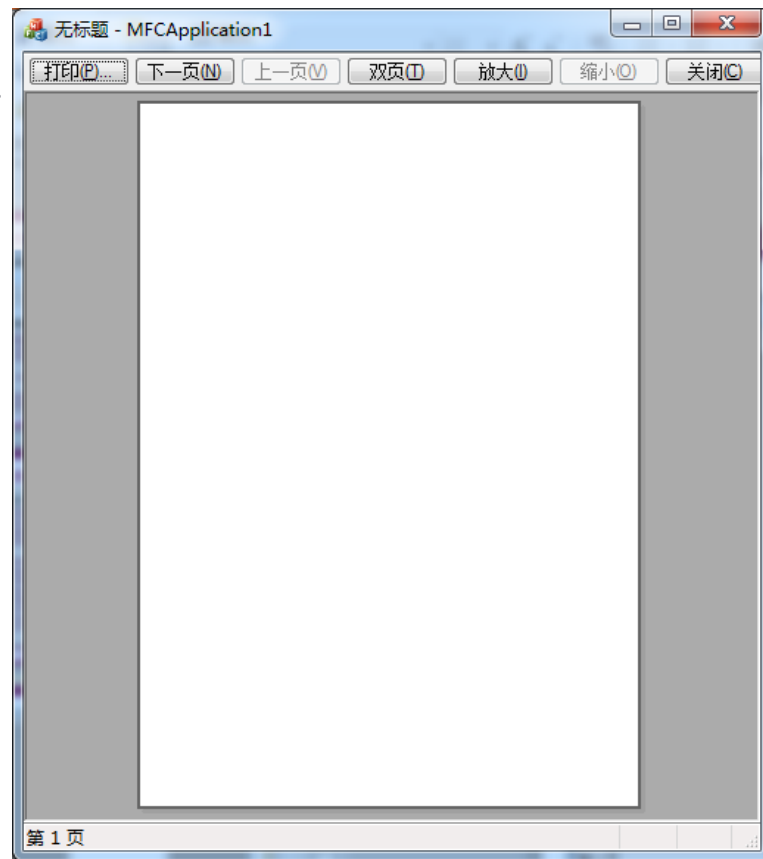
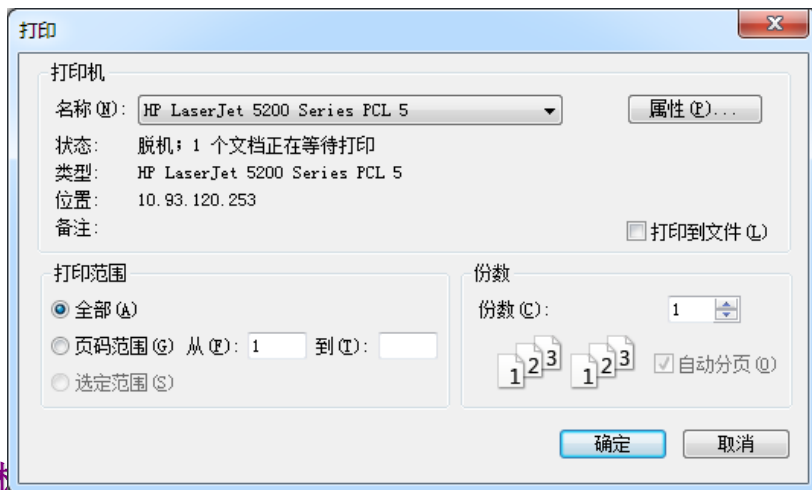
菜单响应
函数

```
CClientDC dc(this);
CFont Font;
LOGFONT LogFont;
COLORREF FontColor;

CFontDialog fontDlg;
if(fontDlg.DoModal() == IDOK) {
    fontDlg.GetCurrentFont(&LogFont);
    Font.CreateFontIndirect(&LogFont);
    FontColor=fontDlg.GetColor();
    dc.SelectObject(&Font);
    dc.SetTextColor(FontColor);
    dc.TextOut(100,100,fontDlg.GetFaceName());
}
```


5、CPrintDialog类

- CPrintDialog类支持Print（打印）和Print Setup（打印设置）对话框，通过这两个对话框用户可以进行与打印有关的操作
- 相关函数
 - GetDefaults：获得默认打印机信息
 - GetCopies：获得打印份数
 - GetFromPage：获得打印起始页
 - GetToPage：获得打印终止页



消息对话框

- 消息对话框是最简单的一类对话框，用来显示信息
- 函数原型

系统程序

```
int AfxMessageBox(           // 返回用户选择按钮的情况  
    LPCTSTR lpszText,        // 消息文本  
    UINT nType=MB_OK,        // 显示按钮和图标  
    UINT nIDHelp=0);
```

```
int MessageBox(              // 返回用户选择按钮的情况  
    LPCTSTR lpszText,        // 消息文本  
    LPCTSTR lpszCaption=NULL, // 对话框标题  
    UINT nType=MB_OK);       // 显示按钮和图标
```

- AfxMessageBox是全局函数，可以用在任何地方
- MessageBox是CWnd的类成员函数，只能在控件、对话框、窗口等一些窗口类中使用

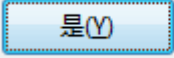
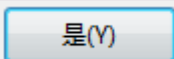
消息对话框

➤ 图标参数

图标类型	含义	符号
MB_ICONHAND MB_ICONSTOP MB_ICONERROR	叉号	
MB_ICONQUESTION	问号	
MB_ICONEXCLAMATION MB_ICONWARNING	感叹号	
MB_ICONASTERISK MB_ICONINFORMATION	信息符号	

消息对话框

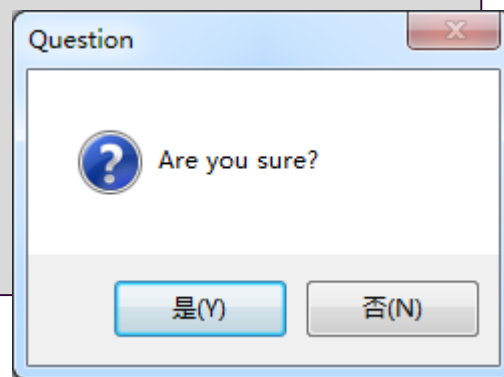
➤ 按钮风格组合参数

按钮类型	显示的按钮
MB_OK	
MB_OKCANCEL	 
MB_RETRYCANCEL	 
MB_YESNO	 
MB_YESNOCANCEL	  

举例：使用消息对话框

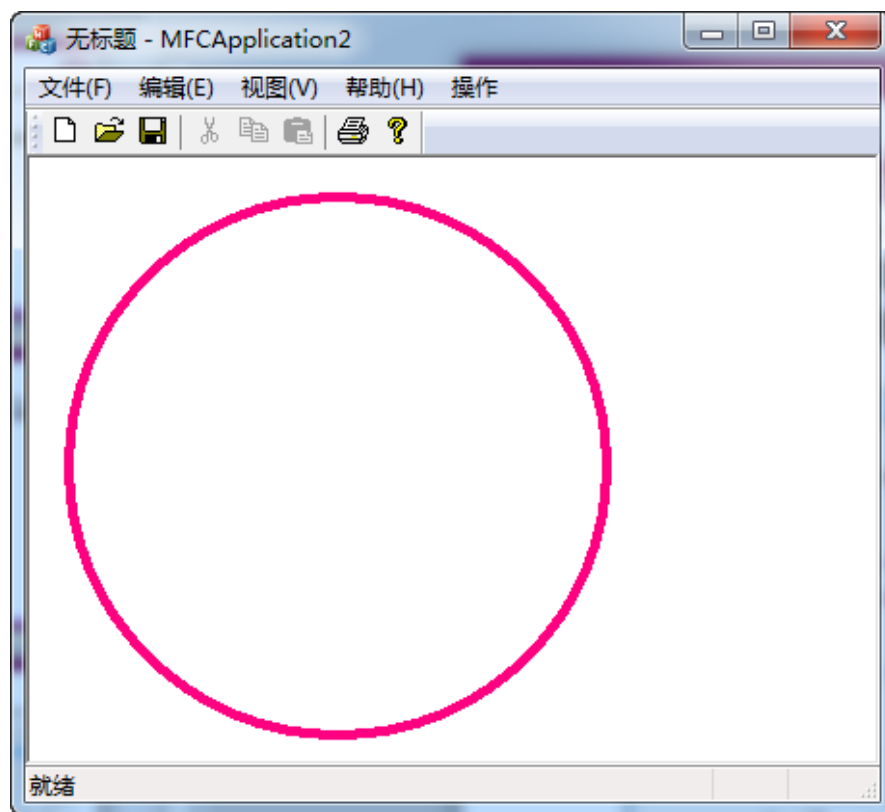
菜单响应
函数

```
CClientDC dc(this);  
int nChoice = MessageBox(_T("Are you sure?"),  
    _T("Question"),MB_YESNO | MB_ICONQUESTION);  
switch(nChoice)  
{  
case IDYES:  
    dc.TextOutW(100,100,_T("I am sure"));  
    break;  
case IDNO:  
    dc.SetTextColor(RGB(255,0,0));  
    dc.TextOutW(100,100,_T("I'm not sure"));  
    break;  
}
```



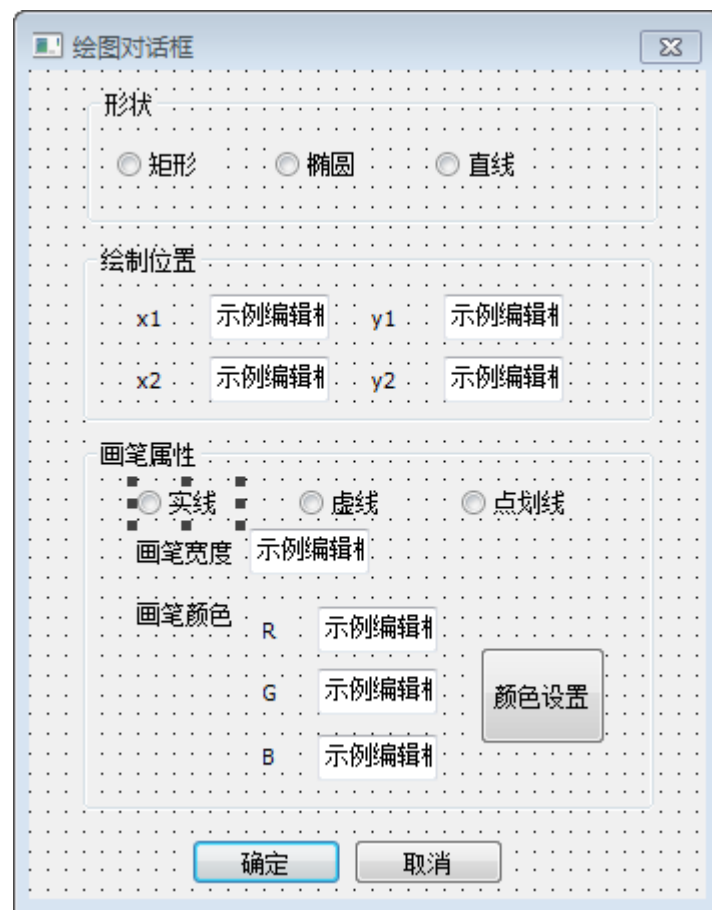
综合实例：绘图程序

- 设计并实现绘图对话框，通过设置绘图形状、绘制位置、画笔属性，实现绘图



综合实例：绘图程序

- 1、用对话框编辑器设计对话框
- 利用格式菜单/工具条排布控件
 - 对齐控件
 - 设置控件大小
 - 标尺和网格
 - Tab order
- 需要设置的属性
 - Caption: 显示标题
 - ID: 根据实际意义修改控件ID
 - Group: 对于单选按钮, 配合 Tab order 设置单选按钮组





系统添加

resource.h

```
#define IDD_DIALOGCOLOR          310
#define IDC_RADIO_RECT            1000
#define IDC_RADIO_ELIP            1001
#define IDC_RADIO_LINE            1002
#define IDC_EDIT_X1               1003
#define IDC_EDIT_Y1               1004
#define IDC_EDIT_X2               1005
#define IDC_EDIT_Y2               1006
#define IDC_RADIO_SOLID           1007
#define IDC_RADIO_DASH            1008
#define IDC_RADIO_DASHDOT         1009
#define IDC_EDIT_PEN_WIDTH        1010
#define IDC_EDIT_PENR             1011
#define IDC_EDIT_PENG             1012
#define IDC_EDIT_PENB             1013
#define IDC_BUTTON_COLOR          1014
#define IDC_STATIC_PEN            1015
```

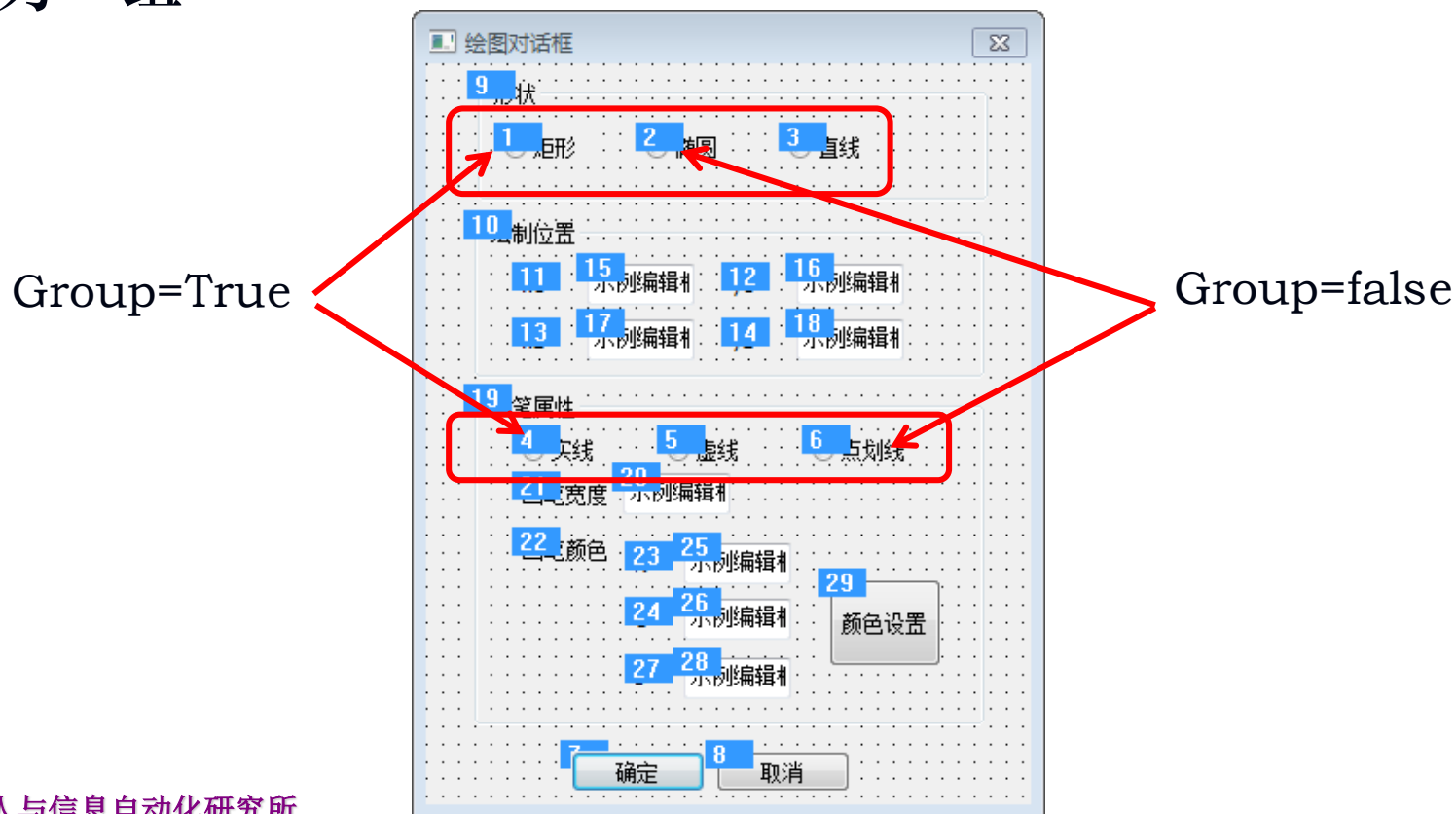



```
IDD_DIALOGCOLOR DIALOGEX 0, 0, 225, 255
STYLE DS_SETFONT | DS_MODALFRAME | DS_FIXEDSYS | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "绘图对话框"
FONT 8, "MS Shell Dlg", 400, 0, 0x1
BEGIN
    CONTROL        "矩形",IDC_RADIO_RECT,"Button",BS_AUTORADIOBUTTON | WS_GROUP,29,24,32,10
    CONTROL        "椭圆",IDC_RADIO_ELIP,"Button",BS_AUTORADIOBUTTON,82,24,32,10
    CONTROL        "直线",IDC_RADIO_LINE,"Button",BS_AUTORADIOBUTTON,135,24,32,10
    CONTROL        "实线",IDC_RADIO_SOLID,"Button",BS_AUTORADIOBUTTON | WS_GROUP,36,128,32,10
    CONTROL        "虚线",IDC_RADIO_DASH,"Button",BS_AUTORADIOBUTTON,90,128,32,10
    CONTROL        "点划线",IDC_RADIO_DASHDOT,"Button",BS_AUTORADIOBUTTON,144,128,40,10
    DEFPUSHBUTTON  "确定",IDOK,54,236,50,14
    PUSHBUTTON     "取消",IDCANCEL,108,236,50,14
    GROUPBOX       "形状",IDC_STATIC,19,6,190,41
    GROUPBOX       "绘制位置",IDC_STATIC,18,54,190,54
    LTEXT          "x1",IDC_STATIC,36,72,9,8
    LTEXT          "y1",IDC_STATIC,114,72,9,8
    LTEXT          "x2",IDC_STATIC,36,91,9,8
    LTEXT          "y2",IDC_STATIC,114,91,9,8
    EDITTEXT       IDC_EDIT_X1,60,69,40,14,ES_AUTOHSCROLL
    EDITTEXT       IDC_EDIT_Y1,138,69,40,14,ES_AUTOHSCROLL
    EDITTEXT       IDC_EDIT_X2,60,88,40,14,ES_AUTOHSCROLL
    EDITTEXT       IDC_EDIT_Y2,138,88,40,14,ES_AUTOHSCROLL
    GROUPBOX       "画笔属性",IDC_STATIC,18,114,190,114
    EDITTEXT       IDC_EDIT_PEN_WIDTH,73,141,40,14,ES_AUTOHSCROLL
    LTEXT          "画笔宽度",IDC_STATIC_PEN,36,144,33,8
    LTEXT          "画笔颜色",IDC_STATIC,36,163,33,8
    LTEXT          "R",IDC_STATIC,78,168,8,8
    LTEXT          "G",IDC_STATIC,78,187,8,8
    EDITTEXT       IDC_EDIT_PENR,96,165,40,14,ES_AUTOHSCROLL
    EDITTEXT       IDC_EDIT_PENG,96,184,40,14,ES_AUTOHSCROLL
    LTEXT          "B",IDC_STATIC,78,207,8,8
    EDITTEXT       IDC_EDIT_PENB,96,204,40,14,ES_AUTOHSCROLL
    PUSHBUTTON     "颜色设置",IDC_BUTTON_COLOR,150,177,42,30
END
```

*.rc文件
系统添加

综合实例：绘图程序

- 设置单选按钮组：对于Tab order连续的单选按钮组，每个设置Group风格的按钮和接下来没有设置Group风格的按钮为一组



综合实例：绘图程序

- 2、用类向导创建对话框类：CMyDlg
- 3、用类向导或属性选项卡中的控件事件，为对话框中的控件与对话框类中的消息处理函数建立映射
 - 响应颜色设置按钮的BN_CLICKED消息：当点击颜色设置按钮，出现颜色对话框获取颜色
 - 响应实线、虚线、点划线单选按钮的BN_CLICKED消息：对于虚线和点划线，画笔宽度为1，因此若选中这两个单选按钮，需使画笔宽度编辑框消失或无效

综合实例：绘图程序

- 4、用类向导添加与控件相关成员变量
 - 绘图形状：int型变量型成员
 - 绘制位置：四个int型变量型成员，正数
 - 画笔属性：int型变量型成员
 - 画笔宽度：控件型成员/int型变量型成员，正数
 - 画笔颜色：三个int型变量型成员，在0~255之间

控件 ID	类型	成员
IDC_BUTTON_COLOR		
IDC_EDIT_PEN_WIDTH	int	m_nPenWidth
IDC_EDIT_PEN_WIDTH	CEdit	m_editPenWidth
IDC_EDIT_PENB	int	m_nColorB
IDC_EDIT_PENG	int	m_nColorG
IDC_EDIT_PENR	int	m_nColorR
IDC_EDIT_X1	int	m_nPosX1
IDC_EDIT_X2	int	m_nPosX2
IDC_EDIT_Y1	int	m_nPosY1
IDC_EDIT_Y2	int	m_nPosY2
IDC_RADIO_RECT	int	m_nShape
IDC_RADIO_SOLID	int	m_nPen
IDC_STATIC_PEN		
IDC_CANCEL		

综合实例：绘图程序

- 4、用类向导添加与控件相关成员变量
 - CMyDlg类构造函数

```
CMyDlg::CMyDlg(CWnd* pParent /*=NULL*/)
: CDialogEx(CMyDlg::IDD, pParent)
, m_nPosX1(20)
, m_nPosX2(300)
, m_nPosY1(20)
, m_nPosY2(300)
, m_nColorB(0)
, m_nColorG(0)
, m_nColorR(0)
, m_nPenWidth(1)
, m_nShape(0)
, m_nPen(0)
{ }
```

添加蓝色
代码

综合实例：绘图程序

- 5、编写控件的消息处理函数
 - 颜色设置按钮的BN_CLICKED消息

```
void CMyDlg::OnBnClickedButtonColor() {  
    UpdateData();  
    COLORREF color;  
    CColorDialog colorDlg;  
    if(colorDlg.DoModal() == IDOK) {  
        color = colorDlg.GetColor();  
        m_nColorR = GetRValue(color);  
        m_nColorG = GetGValue(color);  
        m_nColorB = GetBValue(color);  
        UpdateData(FALSE);  
    }  
}
```

综合实例：绘图程序

➤ 5、编写控件的消息处理函数

- 实线、虚线、点划线单选按钮的BN_CLICKED消息，使画笔宽度编辑框消失。（该方法使宽度编辑框【消失】）

```
void CMyDlg::OnBnClickedRadioSolid()
{
    GetDlgItem(IDC_STATIC_PEN)->ShowWindow(SW_NORMAL);
    m_editPenWidth.ShowWindow(SW_NORMAL);
}
```

```
void CMyDlg::OnBnClickedRadioDash()
{
    GetDlgItem(IDC_STATIC_PEN)->ShowWindow(SW_HIDE);
    m_editPenWidth.ShowWindow(SW_HIDE);
}
```

综合实例：绘图程序

➤ 5、编写控件的消息处理函数

- 实线、虚线、点划线单选按钮的BN_CLICKED消息，使画笔宽度编辑框无效。（该方法使宽度编辑框【无效】）

```
void CMyDlg::OnBnClickedRadioSolid()
{
    m_editPenWidth.EnableWindow();
}
```

```
void CMyDlg::OnBnClickedRadioDash()
{
    m_editPenWidth.SetWindowText(_T("1"));
    m_editPenWidth.EnableWindow(FALSE);
}
```


综合实例：绘图程序

➤ 6、创建对话框模板并显示对话框

➤ C***View类定义中

```
private:  
    CRect rect;           // 绘图区域  
    int m_nShape;         // 绘制形状  
    int m_nPen;           // 画笔类型  
    int m_nPenWidth;      // 画笔宽度  
    COLORREF color;       // 画笔颜色  
    bool m_isDraw;
```

➤ C***View类构造函数中

```
int m_nShape = 0;         // 绘制形状  
int m_nPen = 0;           // 画笔类型  
int m_nPenWidth = 0;      // 画笔宽度  
bool m_isDraw = false;
```

综合实例：绘图程序

➤ C***View类的菜单响应函数

```
CMyDlg Dlg;  
if( Dlg.DoModal() == IDOK) {  
    m_isDraw = true;  
    m_nShape = Dlg.m_nShape; // 绘制形状  
    m_nPen   = Dlg.m_nPen;    // 画笔类型  
    rect.left = Dlg.m_nPosX1; rect.right = Dlg.m_nPosX2;  
    rect.top  = Dlg.m_nPosY1; rect.bottom = Dlg.m_nPosY2;  
    if(m_nPen == 0)                // 实线画笔  
        m_nPenWidth = Dlg.m_nPenWidth;  
    else  
        m_nPenWidth = 1;  
    color = RGB(Dlg.m_nColorR,Dlg.m_nColorG,Dlg.m_nColorB);  
    Invalidate();  
}
```

综合实例：绘图程序

➤ C***View类的OnDraw函数中

```
CPen newPen; CPen * pOldPen;
switch(m_nPen) {
case 0: newPen.CreatePen(PS_SOLID,m_nPenWidth,color); break;
case 1: newPen.CreatePen(PS_DASH,m_nPenWidth,color); break;
case 2: newPen.CreatePen(PS_DASHDOT,m_nPenWidth,color);
        break;
}
pOldPen = pDC->SelectObject(&newPen);
switch(m_nShape) {
    case 0:pDC->Rectangle(rect); break;
    case 1:pDC->Ellipse(rect);    break;
    case 2: pDC->MoveTo(CPoint(rect.left,rect.top));
            pDC->LineTo(CPoint(rect.right,rect.bottom)); break;
}
pDC->SelectObject(pOldPen);
```

总结

- 对话框的基本概念
- 模态与非模态对话框
- 控件的创建和使用方法
- 通用对话框和消息对话框

课上练习

- 阅读第9章第2节和第6节
- 练习例9-1，例9-5，例9-6
- 课后习题9-17